

CSC1029 OO Programming

Lecture 24 – Bubble Sort & Software Testing



What's to come in today's lecture?

- Bubble Sorting
- Software Testing

From the last lecture – Insertion Sort

This algorithm starts with an empty array, inserting elements in their correct location as they are presented.

```
if value is first to be inserted
    insert value at position 0
return
```

```
find insert position for value
```

```
shuffle values ahead of position,
    (if required)
```

```
insert the value
```

Start:

0	1	2	3	4	5	6

Insert: 6

0	1	2	3	4	5	6
6						

Insert: 9

0	1	2	3	4	5	6
6	9					

Insert: 7

0	1	2	3	4	5	6
6	7	9				

Insert: 3

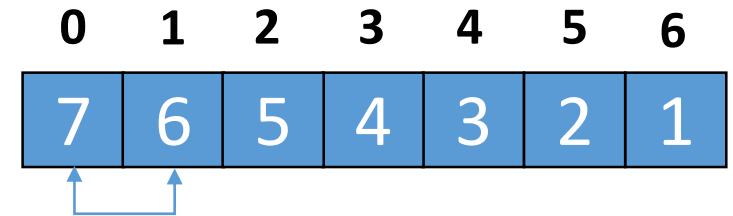
0	1	2	3	4	5	6
3	6	7	9			

Bubble Sort

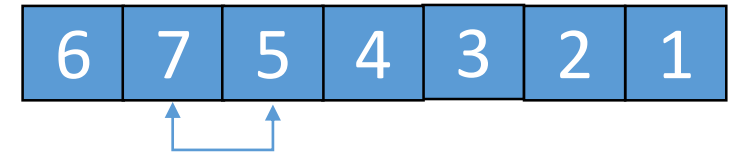
This algorithm passes through the array comparing adjacent elements – swapping if out of order. Such passes continue until no more swaps are required.

```
repeat
  set swaps to 0
  for i = 0 to size-2
    if data[i] > data[i+1] then
      swap data[i], data[i+1]
      swaps++
    end if
  end for
until swaps are 0
```

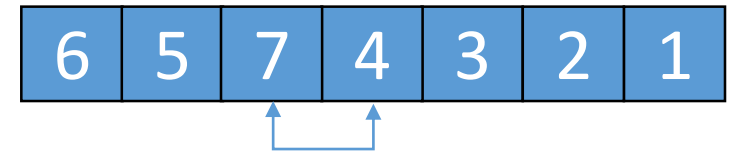
Start:



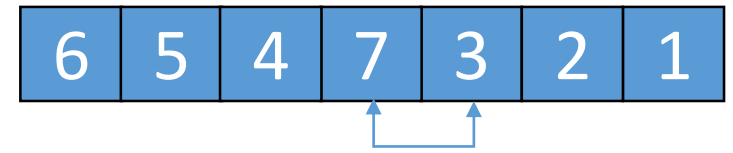
swaps: 1



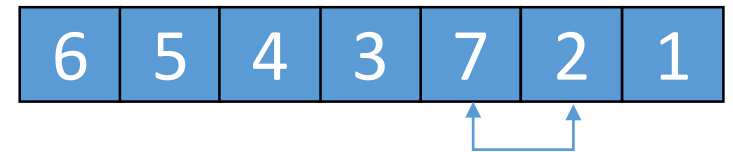
swaps: 2



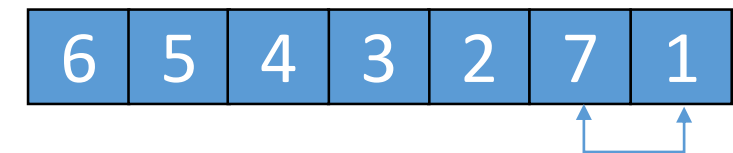
swaps: 3



swaps: 4



swaps: 5



Bubble Sort

This algorithm passes through the array comparing adjacent elements – swapping if out of order. Such passes continue until no more swaps are required.

```
repeat
  set swaps to 0
  for i = 0 to size-2
    if data[i] > data[i+1] then
      swap data[i], data[i+1]
      swaps++
    end if
  end for
until swaps are 0
```

```
public static void bubbleSort(int data[])
{
    int swaps;
    do {
        swaps = 0;
        for(int i=0;i<data.length-1;i++) {
            if (data[i]>data[i+1]) {
                int temp = data[i];
                data[i] = data[i+1];
                data[i+1] = temp;
                swaps++;
            }
        }
    } while(swaps>0);
}
```

Swap depends on comparison.

Bubble Sort – Suitable Test Cases?

- No data to sort?

data



Bubble Sort – Test Case 1

```
private static void bubbleTestCase1() {  
    int data[] = {};  
    System.out.println("+++ Test Case 1: Sort empty array. +++++");  
    System.out.print("Before : ");  
    display(data,0);  
    bubbleSort(data);  
    System.out.print("After  : ");  
    display(data,0);  
    System.out.println("+++ End Test Case +++++\n");  
}
```

```
+++ Test Case 1: Sort empty array. +++++  
Before : No data to display.  
After  : No data to display.  
+++ End Test Case +++++
```

Bubble Sort – Suitable Test Cases?

- Best case – data already in order?



Bubble Sort – Test Case 2

```
private static void bubbleTestCase2() {  
    int data[] = {1,2,3,4,5};  
    System.out.println("+++ Test Case 2: Sort best case. ++++++++");  
    System.out.print("Before : ");  
    display(data,5);  
    bubbleSort(data);  
    System.out.print("After  : ");  
    display(data,5);  
    System.out.println("+++ End Test Case ++++++++\\n");  
}
```

```
+++ Test Case 2: Sort best case. ++++++++  
Before : [1, 2, 3, 4, 5]  
After  : [1, 2, 3, 4, 5]  
+++ End Test Case ++++++++
```

Bubble Sort – Suitable Test Cases?

- Worst case – all elements out of place?

data



Bubble Sort – Test Case 3

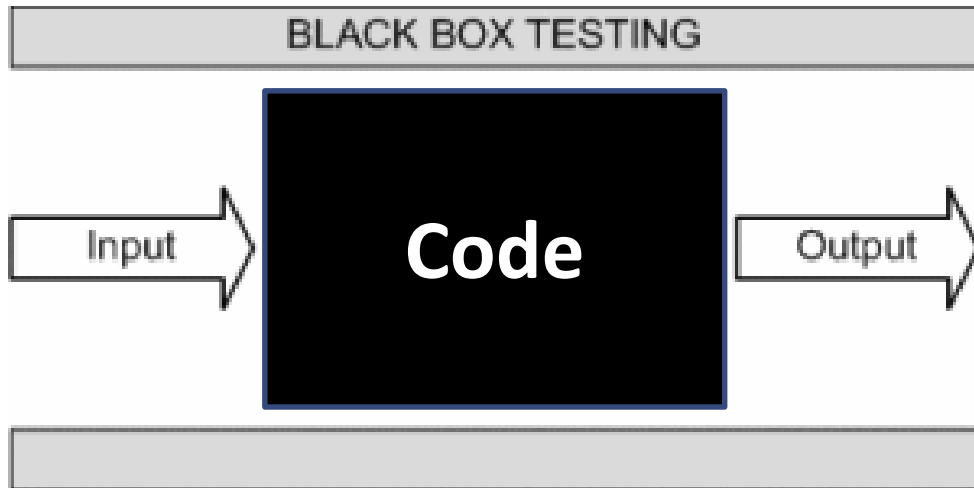
```
private static void bubbleTestCase3() {  
    int data[] = {5,4,3,2,1};  
    System.out.println("+++ Test Case 3: Sort worst case. ++++++++");  
    System.out.print("Before : ");  
    display(data,5);  
    bubbleSort(data);  
    System.out.print("After  : ");  
    display(data,5);  
    System.out.println("+++ End Test Case ++++++++\\n");  
}
```

```
+++ Test Case 3: Sort worst case. ++++++++  
Before : [5, 4, 3, 2, 1]  
After  : [1, 2, 3, 4, 5]  
+++ End Test Case ++++++++
```

Software Testing

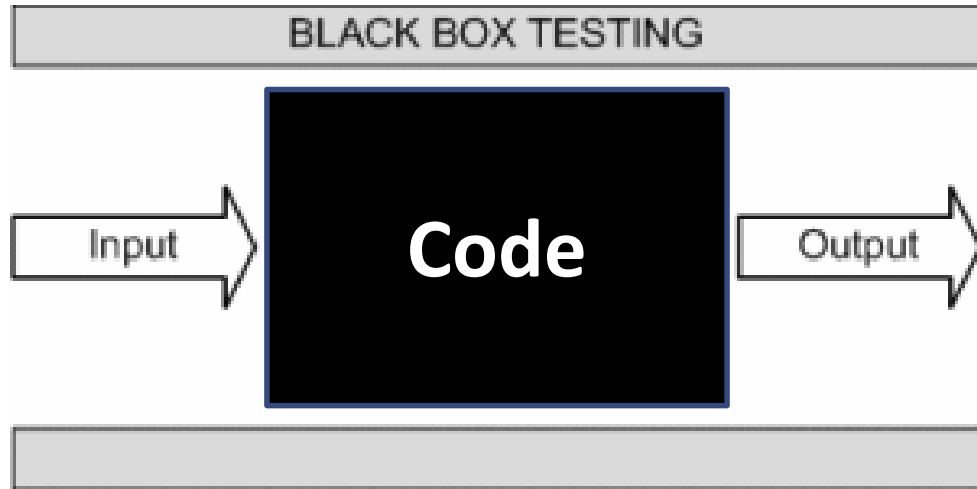
Specification-Based Testing

BLACK BOX TESTING, also known as *Behavioural Testing* or *Specification-Based Testing*, is a *software testing method* in which the internal structure/design/implementation of the item being tested *is not known* to the tester.



This method is named so because the software program, in the eyes of the tester, is like a black box, inside which one cannot see.

Specification-Based Testing



This aim of the testing process is basically to demonstrate (prove?) that a software system behaves as expected with respect to the specification.

This developer is the first tester – will (at least) unit test their own code.

Unit
Testing



Integration
Testing

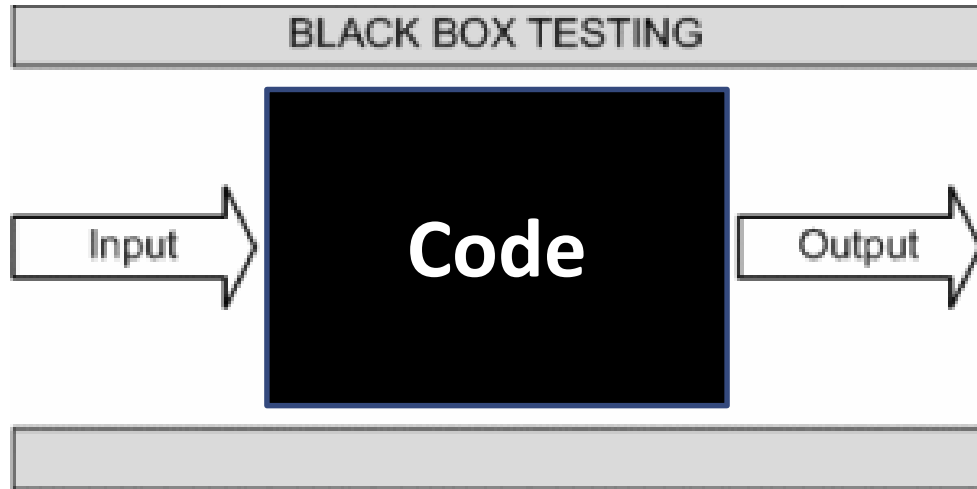


System
Testing



Acceptance
Testing

Specification-Based Testing



The process involves designing suitable **test cases**.

These should be chosen to provide good (100%?) **coverage** of possible responses against input data.

Unit
Testing



Integration
Testing



System
Testing



Acceptance
Testing

Specification-Based Testing

How do we choose test cases to provide good **coverage**?

There are several specification-based or **black-box** techniques to help with this:

- Equivalence Partitioning
 - Boundary Value Analysis
 - Decision Table Testing
 - State Transition Testing
 - Use Case Testing
- We will consider these*

Equivalence Partitioning (EP)

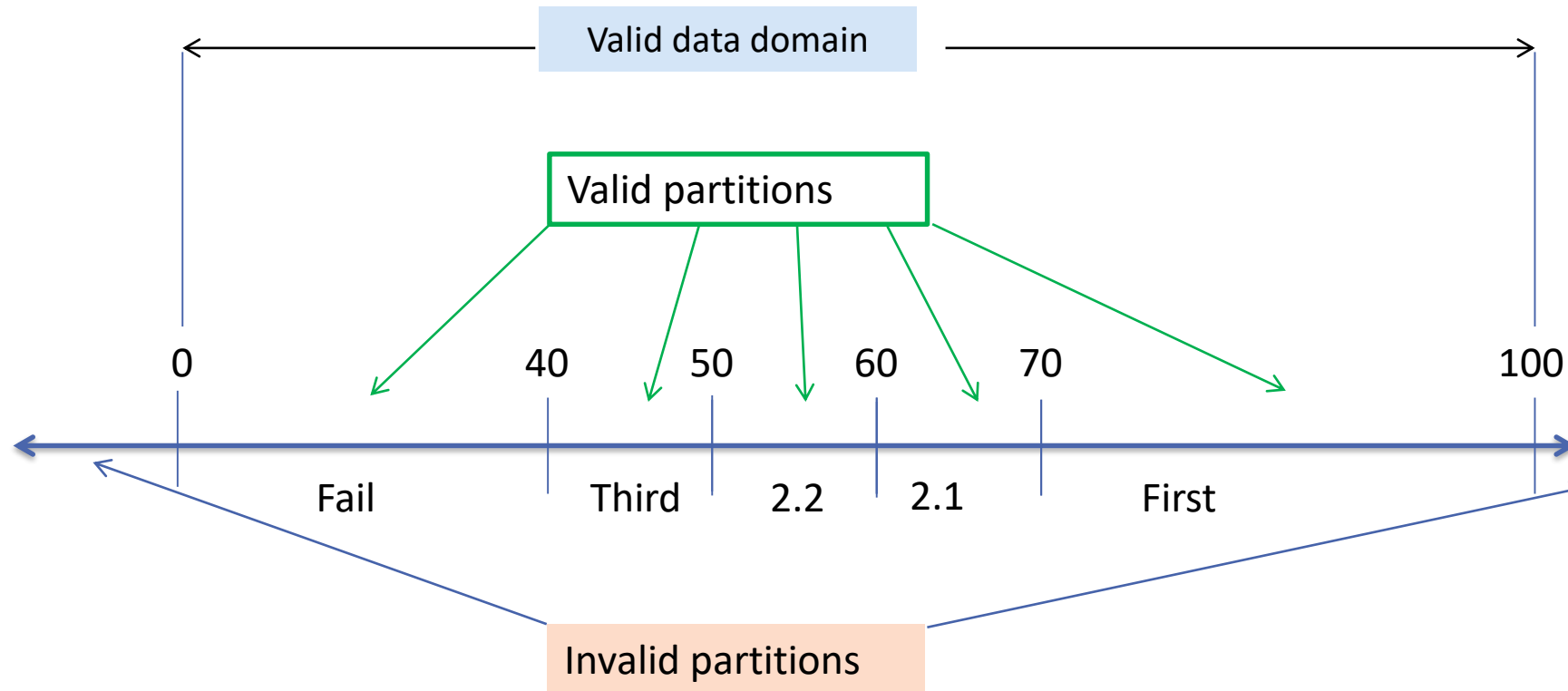
- Can be *applied at any level of testing* and is often a good technique to use first.
- The idea behind this technique is to divide (i.e. to *partition*) a set of *test conditions* into groups or sets that can be considered the same (i.e. the system should handle them equivalently), hence 'equivalence partitioning'.
- *Equivalence partitions* are also known as *equivalence classes* – the two terms mean exactly the same thing.

Equivalence Partitioning (EP)

- In *equivalence-partitioning* technique we need to test **only one case (for a condition)** from **each partition**. (This is because we are assuming that all the cases in one partition will be treated in the same way by the software.)
- If **one case** in a partition works, we **assume all** of the cases in that partition will work, and so there is little point in testing any of the others.
- Similarly, if one of the cases in a partition **does not work**, then we assume that none of the cases in that partition will work so again there is little point in testing any more in that partition.

Equivalence Partitioning Example

Degree Classification – Based on Score



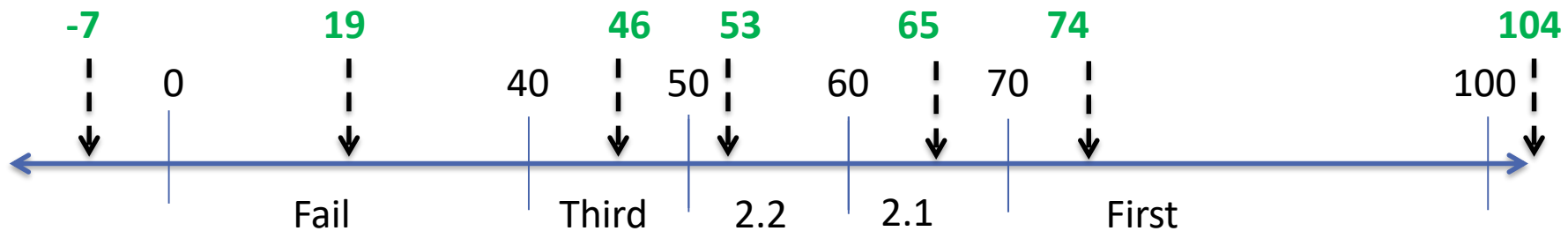
Equivalence Partitioning Example

Degree Classification – Based on Score

Principle :

Only one test value needed from each partition to prove the rule

Example equivalence partitioning test values



Equivalence Partitioning Example

- System for Life Insurance
- Rules :
 - 0-17 Too young “No - minor”
 - 18-65 Acceptable “Yes”
 - 66-99 Too old “No - Retired”
- How would this be coded ...?
- Now how would it be tested using EP to design and develop test cases?

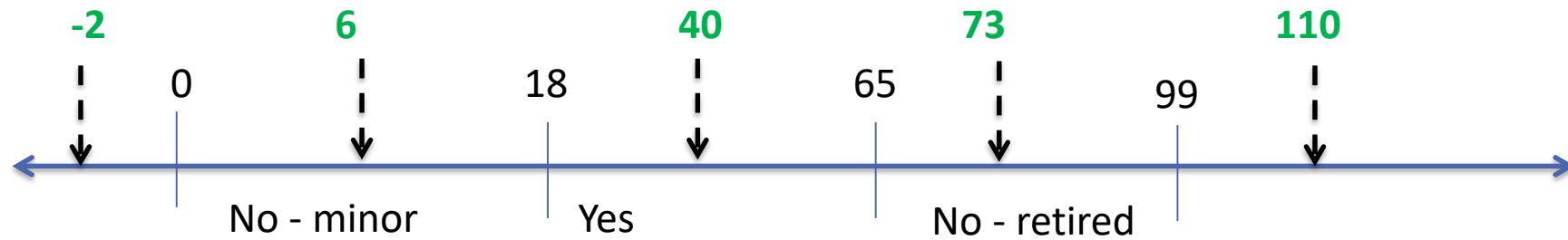
```
public static boolean checkAge(int age) {  
  
    boolean status = false;  
    if (age >= 0 && age <= 17) {  
        System.out.println("No - minor");  
        status = false;  
    } else if (age >= 18 && age <= 65) {  
        System.out.println("Yes");  
        status = true;  
    } else if (age >= 66 && age <= 99) {  
        System.out.println("No - retired");  
        status = false;  
    } else {  
        System.out.println("Invalid range");  
        status = false;  
    }  
    return status;  
}
```

Equivalence Partitioning Example

- Referring to the code ...
- It doesn't need to check every individual value in the range either.
- Question is which value to test for in the range ..
- Rule of thumb ... test the middle one.

```
public static boolean checkAge(int age) {  
  
    boolean status = false;  
    if (age >= 0 && age <= 17) {  
        System.out.println("No - minor");  
        status = false;  
    } else if (age >= 18 && age <= 65) {  
        System.out.println("Yes");  
        status = true;  
    } else if (age >= 66 && age <= 99) {  
        System.out.println("No - retired");  
        status = false;  
    } else {  
        System.out.println("Invalid range");  
        status = false;  
    }  
    return status;  
}
```

Equivalence Partitioning Example



Equivalence Partitioning

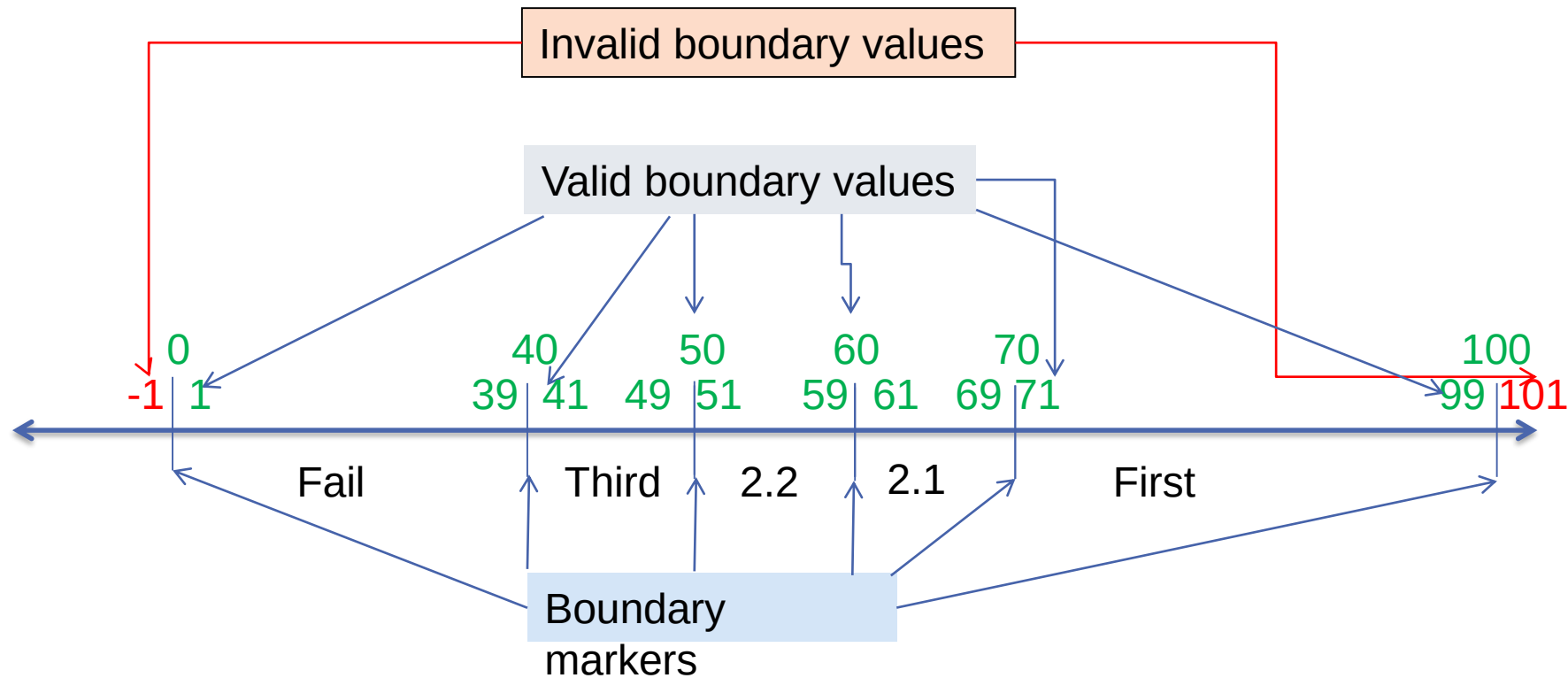
- Partitions may be **valid** or **invalid**.
- EP is a technique used to achieve input and output **coverage**.
- It can be applied to human input, input via interfaces to system or interface parameters in integration testing.

Boundary Value Analysis

- Based on testing at the *boundaries* between *partitions*.
- Here we have *both valid boundaries* (in the valid partitions) and *invalid boundaries* (in the invalid partitions).
- Proves that *the points at which the boundaries change* have been encoded properly
- For each boundary marker the **3** boundary values should be tested

Boundary Value Analysis - Example

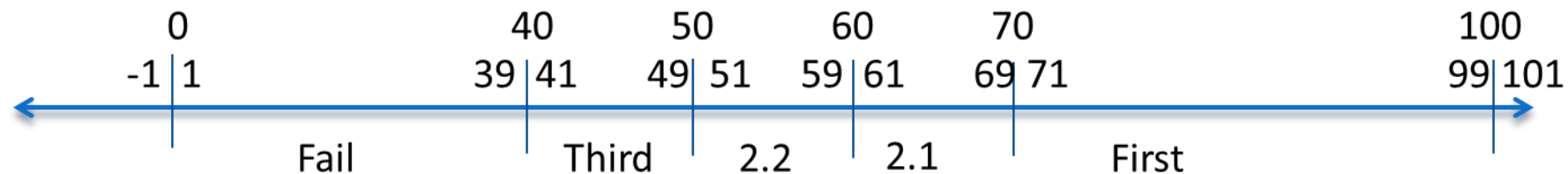
Degree Classification Based on Score



Boundary Value Analysis - Example

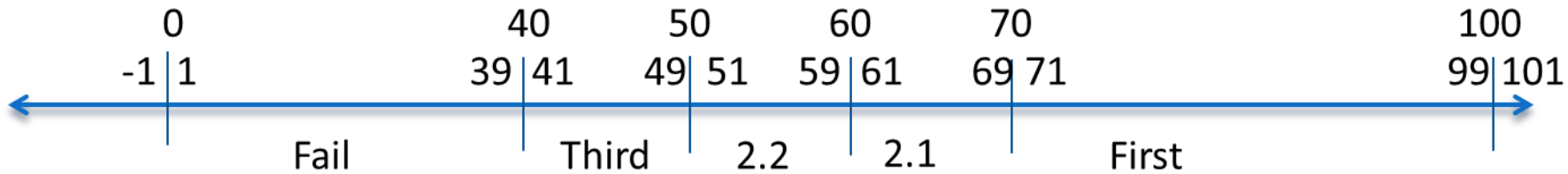
Degree Classification Based on Score

- Select the boundary values (For each boundary marker the **3** boundary values should be tested)
- Then select one value in the valid and one value in the invalid portion.
- Proves that the points at which the code changes have been encoded properly

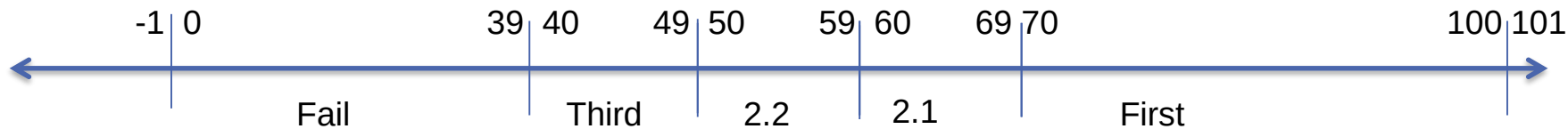


Boundary Value Analysis

- Using three values - 18 tests required



- Alternative (2 boundary values) - 12 tests required



Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Test case ID

TCase_1

Each Test Case must have a unique identifier.

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		



Objective
Check if a valid username and password enables the user to login

What is the Test Case aiming to achieve?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Preconditions
On the login page - and have a registered user account

What is the situation before a Test Case is executed?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Test data
Username : aidan.mcgowan@qub.ac.uk password : aidanaidan

What input data is required for the Test Case?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Expected Result

The user is logged in

What is the expected outcome for the Test Case?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

**Test
completion date**

Record the completion date for the Test Case

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Status

Passed

Did the test pass or fail?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Tester
AMcG

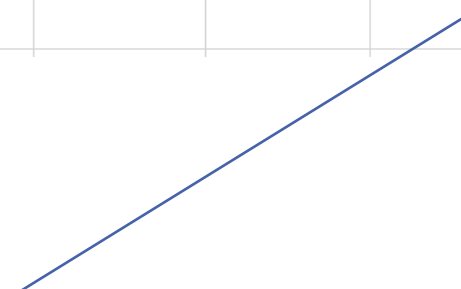
Who conducted the test?

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

Record the defects?



Defect ID	Comments

Testing Template

Available on Canvas ...

Test case ID	Objective	Preconditions	Test data	Expected Result	Test completion date	Status	Tester	Defect ID	Comments
TCase_1	Check if a valid username and password enables the user to login	On the login page - and have a registered user account	Username : aidan.mcgowan@qub.ac.uk password : aidanaidan	The user is logged in		Passed	AMcG		

A very common question: ***How much testing is sufficient for Assessment 2?***

Adopt a ***sensible*** approach:

- Unit test the object classes
- Test the behaviour of the system as a whole
- Don't get carried away – select test cases to cover the core features/behaviour



What's Next?

Fundamental Data Structures